Problem Description

This document details an algorithm for an altitudinal airbrake system for a rocket. After motor burnout, the airbrake system deploys from the side of the rocket, increasing drag and thus slowing the rocket down. This system allows the rocket to reach a desired altitude.

Algorithm Choice

Model Predictive Control (MPC) is used to control the airbrakes. Given a cost over the desired timestep, MPC uses a quadratic programming algorithm to determine the best airbrake deployment throughout the time horizon. It then executes on the first of these time steps and recalculates for the next.

MPC was chosen over Proportional Integral Derivative (PID) and Linear Quadratic Regulator (LQR) due to its ability to better predict how current decisions impact future planning. With a large enough time window, too much or too little airbrake deployment at the start can be shown to have outstanding effects near the end.

Rocket Kinematics

Consider a rocket launched with upward velocity v_0 at height z_0 just after motor burnout. We will only be considering vertical velocity and position from now onward. The force on the rocket is given by

$$F = -mg - F_{\text{Drag}} \tag{1}$$

Where m is the mass of the rocket *after burnout* and $g = 9.81\,\text{m/s}^2$. The force due to drag can be broken up into two separate components: drag from the rocket itself, and drag from the airbrakes. Given this, we can say

$$\begin{aligned} F_{\text{Drag}} &= \frac{1}{2}\rho C_{D_R} A_R v^2 + \frac{1}{2}\rho C_{D_A} A_A v^2 \\ &= \frac{1}{2}\rho\big(C_{D_R} A_R + C_{D_A} A_A\big)v^2 \end{aligned} \tag{2}$$

where the following terms are used:

$$\begin{aligned} \rho &\equiv \text{Air density} \\ C_{D_R} &\equiv \text{Rocket coefficient of drag} \\ C_{D_A} &\equiv \text{Airbrakes coefficient of drag} \\ A_R &\equiv \text{Rocket cross sectional surface area} \\ A_A &\equiv \text{Airbrakes cross sectional surface area} \end{aligned} \tag{3}$$

In reality, many of these terms depend on environmental constants. A simplified model (which we will be using) for air density depends on height, and the cross sectional surface

area of the airbrakes depend on the airbrake opening amount we will call α. Thus the drag force is the function

$$F_{\text{Drag}}[z, \alpha] = \frac{1}{2}\rho[z]\Big(C_{D_R}A_R + C_{D_A}A_A[\alpha]\Big)v^2 \tag{4}$$

Newton's second law gives

$$m\ddot{z} = F \tag{5}$$

and thus

$$\begin{aligned} m\ddot{z} &= -mg - \frac{1}{2}\rho[z]\Big(C_{D_R}A_R + C_{D_A}A_A[\alpha]\Big)v^2 \\ \frac{dv}{dt} &= -g - \frac{1}{2m}\rho[z]\Big(C_{D_R}A_R + C_{D_A}A_A[\alpha]\Big)v^2 \end{aligned} \tag{6}$$

Given environmental parameters z_i, v_i, and α_i, after some small timestep Δt, the next set of environmental parameters is given by

$$v_{i+1} = v_i + \left(-g - \frac{1}{2m}\rho[z_i]\Big(C_{D_R}A_R + C_{D_A}A_A[\alpha_i]\Big)v_i^2\right)(\Delta t) \tag{7}$$

$$z_{i+1} = z_i + v_i(\Delta t) \tag{8}$$

The apogee h can thus be calculated as

$$h = \max_N \left(z_0 + \sum_{i=0}^{N} v_i(\Delta t)\right) \tag{9}$$

In fact, the calculation of h takes several parameters. They are

$$\begin{aligned} z_0 &\equiv \text{Height after motor burnout} \\ \Delta t &\equiv \text{Time step} \\ \vec{\alpha} &\equiv \text{List of airbrake deployment amounts for all steps from } [0, N] \end{aligned} \tag{10}$$

Thus h is the function

$$h[z_0, \Delta t, \vec{\alpha}] = \max_N \left(z_0 + \sum_{i=0}^{N} v_i[v_i, z_i, \alpha_i](\Delta t)\right) \tag{11}$$

However, because it is usually difficult to run MPC on a varying set (in our case, $\vec{\alpha}$) in a single optimization, we will likely set the horizon larger than needed and discard the part beyond apogee.

Defining the Cost Function

Model Predictive Control needs a cost function to determine how close a given airbrake deployment vector $\vec{\alpha}$ is to the target apogee. Several options for a cost function exist, but the one being used has the form

$$J = w_A\left(\hat{h} - h\right)^2 + w_D \sum_{i=1}^{N} \left(\alpha_i - \alpha_{i-1}\right)^2 \tag{12}$$

The first term is the expected apogee $\hat{h}$ minus the predicted apogee h, and the second term ensures the difference in airbrake deployment between subsequent timesteps is minimized. w_A and w_D are the apogee and deployment weights, which can be tuned to favor one term over the other and ensure stricter adherence. This cost function is quadratic as is expected for MPC.

Running Model Predictive Control

Setup

The following functions should be created to allow MPC to operate:

1. `next_height(prev_height, prev_velocity, timestep)` - Calculates the height of the rocket at the next timestep; given by Equation 8
2. `next_velocity(prev_velocity, air_density, airbrake_deployment_amount, timestep)` - Calculates the velocity of the rocket at the next timestep; given by Equation 7
3. `cost_function(current_height, airbrake_deployment_amount_vector)` - For the current height and list of airbrake deployments, calculates how well the airbrake deployments conform to constraints

Algorithm

The MPC algorithm requires the following steps:

1. Get the current height from an environmental sensor
2. Using a quadratic programming solver, compute the minimum $\vec{\alpha}$ given the cost function
3. Execute only on α_i
4. Repeat until apogee is reached